8. Attention and Transformers

1. Attention과 Transformer — RNN에서 출발

- **Attention**: 벡터 집합을 입력받아, 필요한 정보를 가중합으로 모으는 연산.
- **Transformer**: Attention을 중심으로 구성한 신경망 구조.

RNN을 이용한 기계 번역 → Attention을 일반적인 연산으로 확장.

Seq2Seq — Encoder–Decoder

예시: 영어 **"we see the sky" → 이탈리아어 "vediamo il cielo"**

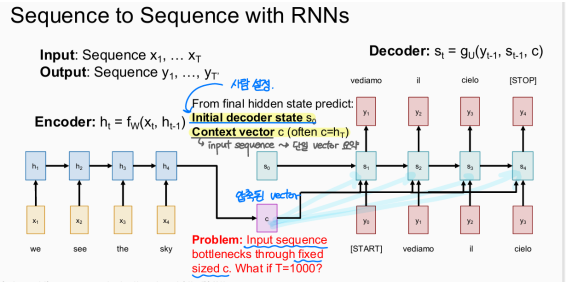
- **Encoder**: 입력을 순서대로 읽으며 hidden state 갱신.

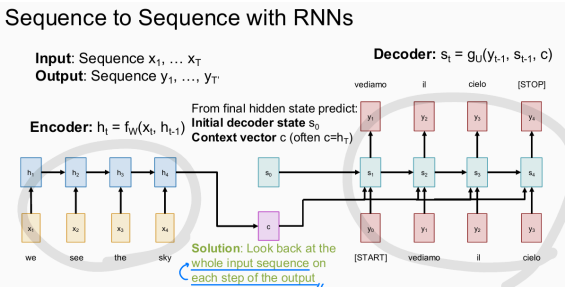
  마지막 hidden state ⇒ **Decoder의 초기 상태 + context vector**

- **Decoder**: 이전 출력, 이전 hidden state, context vector를 이용해 다음 출력을 생성. `[START]` 시작 → `[STOP]`까지 반복.

문제: 입력 전체를 고정된 크기의 context vector C하나에 담아야 함. → **정보 병목**.

⇒ 출력 단어를 만들 때마다 **입력 전체를 다시 살펴보자.**





---

2. Seq2Seq with RNNs and Attention

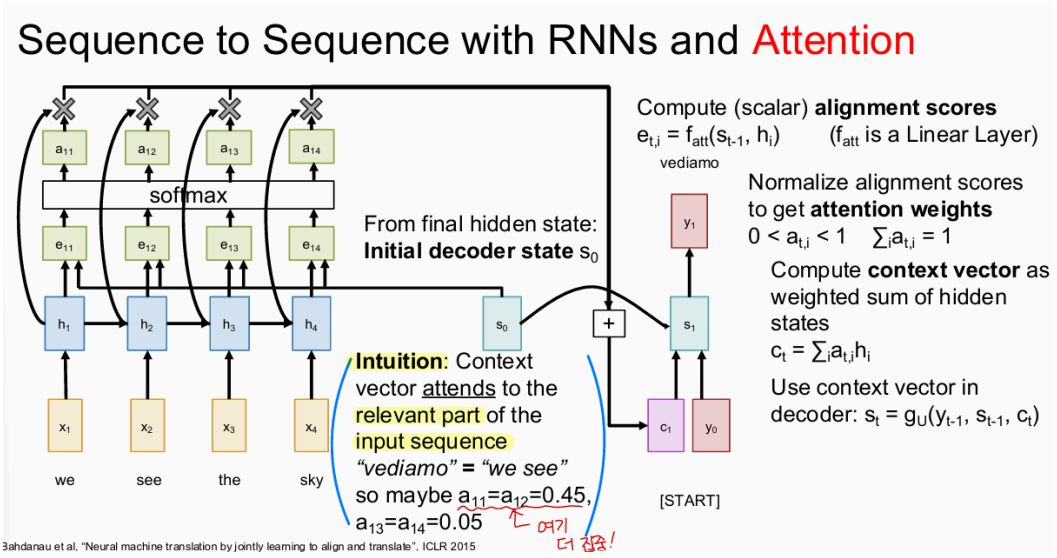
⇒ 출력 시점마다 다른 context vector 사용 (고정된 `c` 하나 → 출력 시점별 `c_t`**.)**

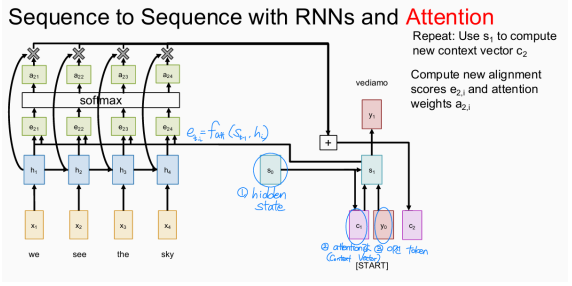
- Encoder의 hidden state들을 보관하고, Decoder가 현재 필요한 부분을 선택적으로 참조.
- 입력 정보를 단일 벡터에 압축하는 병목을 완화.

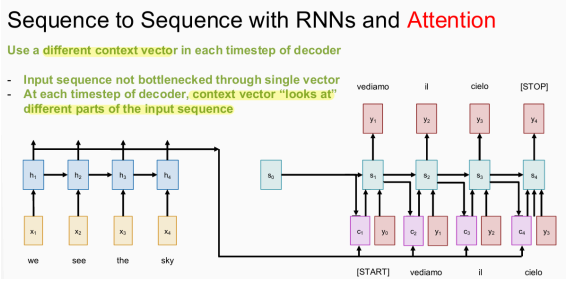
한 번의 출력 생성 과정

1. **Alignment score 계산:** 이전 Decoder 상태 + 각 Encoder hidden state가 얼마나 관련 있는지 점수화.
2. **Softmax:** 각 입력 위치를 얼마나 참조할지 attention weight 계산.
3. **Context vector 계산:** Encoder hidden state들을 attention weight에 따라 가중합.
4. **Decoder 갱신:** 새 context vector를 이용해 다음 상태와 출력 생성.

⇒ 같은 입력을 보더라도 **무슨 단어를 출력하는지에 따라 참조 비중이 바뀜**.





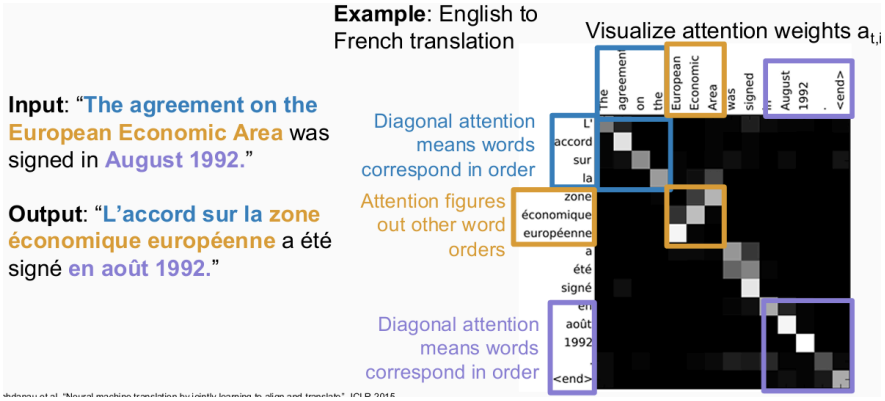


- $e_{t,i}$: 출력 시점 t에서 입력 위치 i와의 관련성 점수.
- $a_{t,i}$: 해당 입력을 참조하는 비중. **양수이며, 입력 위치 전체에 대한 합은 1**.
- c_t: **현재 출력 시점에 맞춰 모은 입력 정보**.



특징

- Decoder 자체: 여전히 LSTM·GRU 등의 RNN unit.
- Attention weight 학습 과정
  - 어느 단어에 집중해야 하는지 **별도의 정답을 주지 않음**.
  - 점수 계산 → softmax → 가중합이 모두 미분 가능.
  - 번역 과제의 손실에서 **전체 경로로 backpropagation** → 학습.
- Attention 시각화 — 단어의 대응 관계
  - 밝은 칸: 해당 출력이 그 입력을 많이 참조.
  - **대각선에 가까운 패턴**: 입력과 출력의 단어 순서가 대체로 대응.
  - **대각선에서 벗어난 패턴**: 언어에 따른 어순 변화 반영.



3. Attention Layer

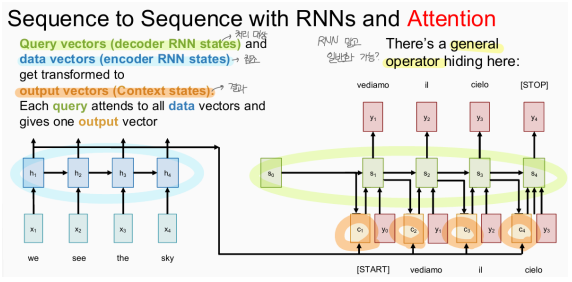
기본 계산

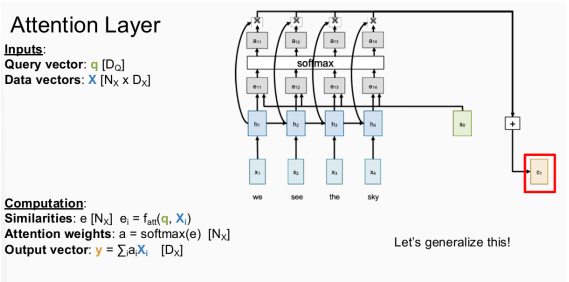
Query 하나가 모든 Data vector 참조 → Output 1개 생성.

번역 모델의 요소	일반적인 역할
Decoder hidden state	**Query**: 현재 어떤 정보를 찾는가
Encoder hidden states	**Data vectors**: 참조할 정보
Context vector	**Output**: Query에 맞춰 모은 정보

과정: Query와 각 데이터의 관련성 계산 → softmax → 데이터 가중합

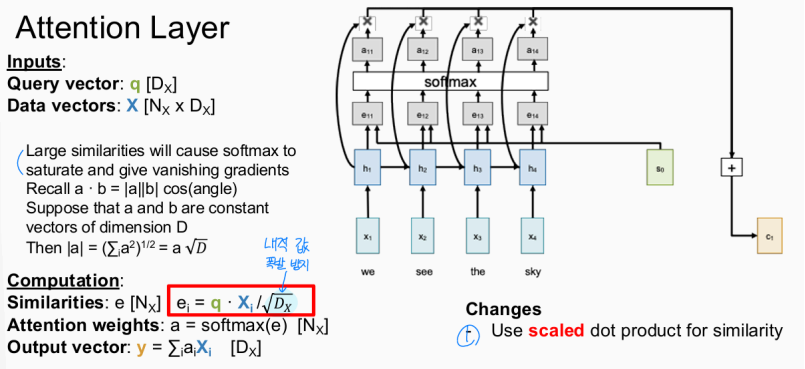
- input data N개 → attention score N개.
- Softmax: 참조 비중으로 변환.
- output: 그 비중으로 data를 섞은 vector.





확장 ① Scaled Dot Product

Attention score = Query & Data vector의 내적



score 조절 ← 학습 안정성. Gradient Vanishing 방지.

확장 ② Query 여러 개를 동시에 처리

전제: Query가 사전에 주어짐

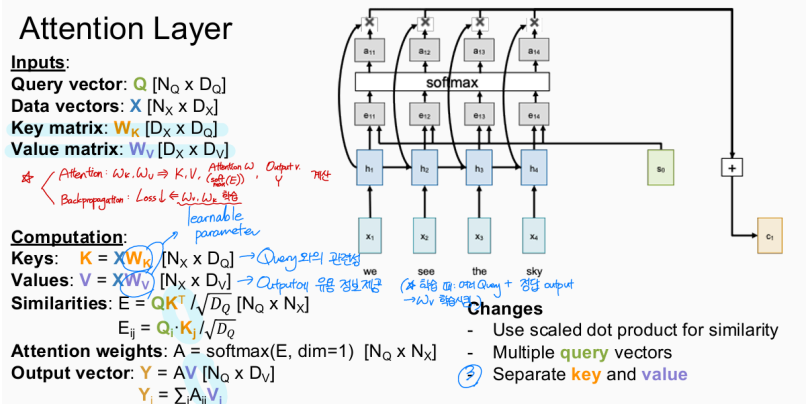
- Query → matrix Q 묶음.
- 모든 Query와 모든 Data vector 사이의 점수를 한 번에 계산

  ⇒ 각 Query마다 별도의 attention 분포와 출력을 얻음.
- Attention score: Q * X 개
  출력: Q개

확장 ③ X → Key, Value 분리

같은 데이터에서 두 종류의 표현을 학습.

- **Key**: Query와의 관련성을 계산할 때 사용.
- **Value**: 실제 출력에 담을 정보.
- W_K, W_V: 데이터를 각각 Key·Value로 바꾸는 학습 가능한 가중치.



계산 흐름

X → Key·Value 생성 → Query–Key 비교 → attention weight → Value 가중합.

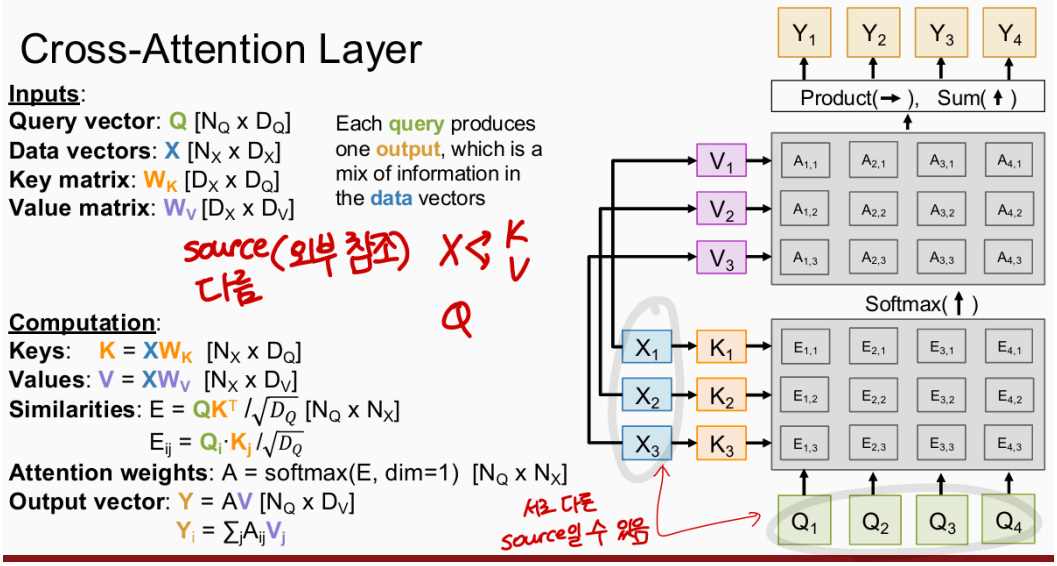
⇒ **어디를 참조할지 판단하는 표현과, 가져올 정보의 표현을 분리**.

(+) Cross-Attention

Query와 Key·Value가 **서로 다른 입력 source**에서 오는 경우.

(예시) translation
- Query ← Decoder
- Key·Value ← Encoder 쪽 정보

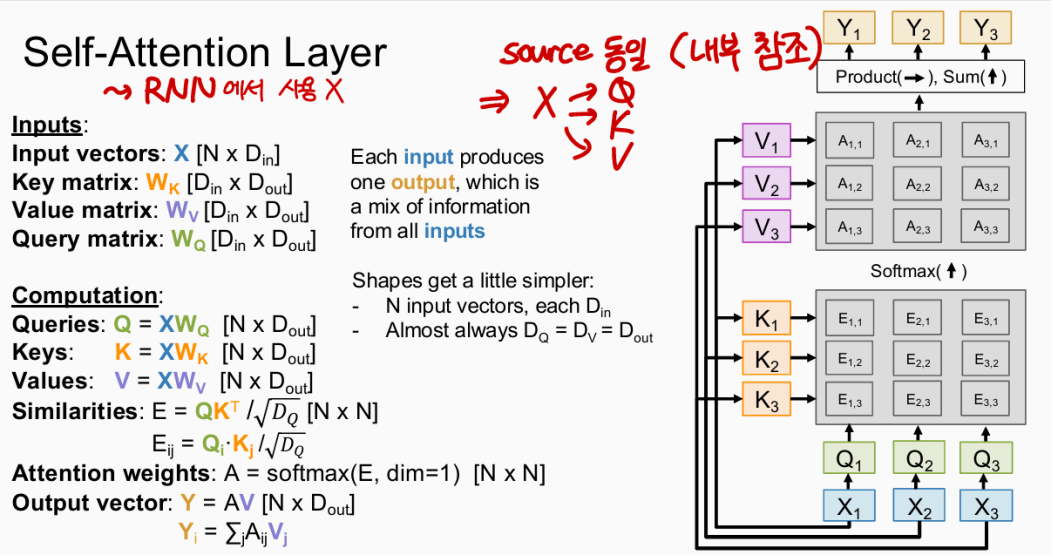
- Query 1개 당 output 1개 생성. 각 output ← 참조 데이터의 정보가 섞임.
- Softmax: **각 Query에 대해 모든 Key 방향으로** 적용. 각 Query의 참조 비중 합 = 1.



4. Self-Attention Layer

같은 입력에서 **Query·Key·Value** 모두 생성

- 일반적으로 Query·Key·Value의 차원을 같게 설정.
- **각 입력의 출력에 다른 입력들의 정보가 반영됨.**



Permutation Equivariant — 순열 등변성

입력 순서를 바꾸면 출력 순서도 똑같이 바뀜.

⇒ 위치 정보가 없는 Self-Attention은 **벡터 집합**을 처리.

Positional Encoding — 순서 정보

문제: 문장에서는 같은 단어라도 **어디에 놓이는지** 중요.

⇒ 입력 벡터 + 위치 벡터 → Self-Attention

---

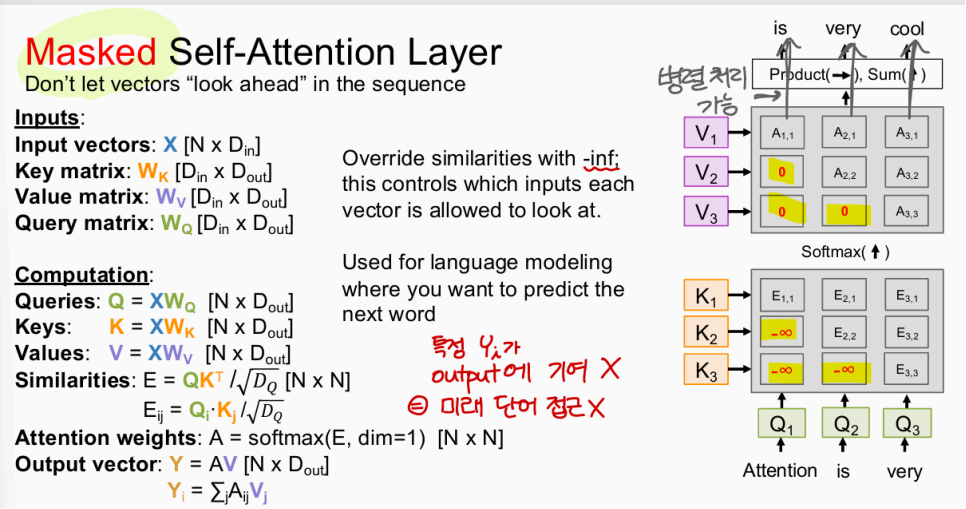
5. Masked Self-Attention

문제: 다음 단어를 예측할 때 미래를 보면 안 됨

⇒ 다음 토큰 예측에 맞는 **인과적 정보 흐름**을 유지.

처리 방법

- 참조하면 안 되는 위치의 **attention score: $-\infty$로 설정**
- Softmax → 해당 위치의 attention weight = 0.
- 각 위치는 **자기 위치와 이전 위치까지** 참조.



---

6. Multiheaded Self-Attention

⇒ 서로 다른 Attention을 병렬로 학습

같은 입력에 대해 **H개의 독립적인 Self-Attention** 실행. 각각을 **head**라고 부름.

- Head마다 자기 Query·Key·Value 가중치를 가짐.
- 따라서 입력 사이의 관계를 서로 다른 방식으로 학습할 수 있음.
- 각 입력 위치에서 나온 **H개 head의 출력을 이어 붙임**.
- **Output projection**으로 여러 head의 정보를 결합.

입력 → H개 head → 출력 연결 → Output projection

실제로는 head를 하나씩 실행하지 않고 **batched matrix multiplication으로 병렬 처리**.

---

7. Self-Attention is Four Matrix Multiplies

Self-Attention의 주요 계산은 **행렬곱 네 번**으로 정리.

단계	연산	역할
1. QKV Projection	입력 × QKV 가중치	Query·Key·Value 생성 후 head별 분리
2. QK Similarity	$QK^\top$	모든 Query–Key 쌍의 관련성 계산
3. V-Weighting	AV	Attention weight로 Value 가중합
4. Output Projection	연결한 출력 × W_O	Head 정보 결합

입력이 길어지면 — 계산량과 메모리

입력 개수 N이 늘어나면, 모든 입력 쌍을 비교해야 함.

- **계산량:** $O(N^2)$
- Attention 행렬을 전부 저장하는 기본 방식의 **메모리:** $O(N^2)$
- Head가 H개면 attention weight의 크기는 $H \times N \times N$.

강의 예시: N=100,000, H=64에서는 attention weight 저장만 약 **1.192 TB**.

⇒ 병렬화하기 좋아도, **긴 입력에서는 행렬 크기가 부담**.

FlashAttention

- QK 관련성 계산과 Value 가중합을 연계해 처리.
- **전체 attention 행렬을 메모리에 저장하지 않음**.
- 강의의 복잡도 비교에서 메모리 사용량은 $O(N)$.
- 더 긴 입력을 처리할 수 있게 함.

구분: 메모리 사용량을 줄여도, 모든 입력 쌍을 다루는 계산량은 $O(N^2)$.

---

8. 3 Ways of Processing Sequences

방식	처리 대상	장점	제약
RNN	순서가 있는 1차원 시퀀스	길이에 대해 계산·메모리가 선형으로 증가	이전 hidden state가 필요해 순차 계산
Convolution	다차원 격자	여러 위치의 출력을 병렬 계산	넓은 범위를 보려면 여러 층을 쌓아 수용 영역 확대
Self-Attention	벡터 집합	한 층에서 모든 입력을 직접 참조, 행렬곱으로 병렬화	입력 길이에 대해 계산량이 제곱으로 증가

- **RNN:** 정보가 hidden state를 따라 전달.
- **Convolution:** 가까운 영역부터 조합하고, 깊이를 통해 범위를 확장.
- **Self-Attention:** 각 출력이 모든 입력에 직접 접근.

이 비교에서 **Attention 중심의 모델 구조**로 연결 → *Attention Is All You Need*.

---

9. The Transformer

- **Transformer Block:** 입력 → Multihead Self-Attention → 잔차 연결 → LayerNorm → MLP → 잔차 연결 → LayerNorm → 출력
- Transformer: Transformer Block * n개

① Self-Attention — 벡터 사이의 정보 교환

- 모든 입력 벡터가 서로 참조.
- 각 위치의 표현에 다른 위치의 정보 반영.

② Residual Connection — 입력을 더해 전달

- Attention 출력에 원래 입력을 더함.
- MLP 부분에도 같은 방식의 잔차 연결 적용.

③ Layer Normalization

- **각 벡터 내부의 특징 차원**을 기준으로 평균·표준편차를 계산해 정규화.
- 학습 가능한 scale `\gamma`, shift `\beta` 적용.
- 각 토큰 벡터를 독립적으로 처리.

④ MLP / FFN

- 각 위치에 같은 MLP를 독립적으로 적용.
- 전형적인 2층 구조: **D** → **4D** → **D**.
- 벡터 내부의 표현을 변환.

역할 구분

구성 요소	처리 범위
Self-Attention	서로 다른 벡터 사이의 상호작용
LayerNorm	각 벡터 내부
MLP	각 벡터 내부

⇒ 이 블록에서 **벡터 간 정보 교환은 Self-Attention이 담당**.

주요 계산은 **Attention의 행렬곱 4번 + MLP의 행렬곱 2번 = 6번**. 병렬 처리와 규모 확장에 적합.

블록을 반복해서 쌓기

같은 구조의 Transformer Block을 여러 층 쌓음. **블록 구조가 같다는 뜻이며, 층마다 가중치는 별도**.

강의에 제시된 규모 비교:

모델	블록 수	모델 차원 D	Head 수 H	시퀀스 길이 N	파라미터
원 논문	12	1,024	16	512	213M
GPT-2	48	1,600	25	1,024	1.5B
GPT-3	96	12,288	96	2,048	175B

⇒ 기본 구성은 유지하면서 **깊이·차원·head 수·입력 길이**를 확대.

---

10. Transformers for Language Modeling — LLM

토큰을 벡터로 바꾸고, 다음 토큰을 예측

토큰 → Embedding → Transformer Blocks → 어휘별 점수 → 다음 토큰 예측

1. **Embedding Matrix**
   - 어휘 크기가 V, 모델 차원이 D이면 **V × D** 크기.
   - 토큰에 해당하는 D차원 벡터를 조회.
2. **Transformer Blocks**
   - 내부에서 **Masked Self-Attention** 사용.
   - 각 위치가 다음 토큰을 예측할 때 미래 입력을 보지 못하게 함.
3. **Projection Matrix**
   - **D × V** 크기.
   - 각 출력 벡터를 어휘 전체에 대한 V개 점수로 변환.
4. **학습**
   - Softmax와 cross-entropy loss.
   - 각 위치에서 **다음 토큰**을 맞히도록 학습.

---

11. Tweaking Transformers — 구조 개선

11-1. Pre-Norm Transformer

앞에서 구성한 블록은 **잔차 덧셈 뒤에 LayerNorm**이 오는 Post-Norm 방식.

Post-Norm에서 짚는 문제

- 잔차 가지의 출력이 0이어도 전체 출력은 `LN(x)`.
- 입력 x를 그대로 전달하는 **항등 경로가 정규화를 통과**.

Pre-Norm의 변경

- LayerNorm을 Attention과 MLP **앞으로 이동**.
- 정규화가 잔차 가지 내부로 들어감.
- 입력이 그대로 흐르는 경로 유지.
- **학습 안정성이 좋아짐**.

11-2. RMSNorm

LayerNorm을 **Root-Mean-Square Normalization**으로 교체.

- 벡터 원소들의 제곱 평균을 이용해 크기 조절.
- LayerNorm처럼 평균을 빼는 과정 없이 **RMS로 나눔**.

- 학습 가능한 scale 적용.

강의와 필기에서는 **학습 안정성의 개선**을 강조.

11-3. SwiGLU MLP

기존 MLP는 **확장 → 활성화 → 축소**.

SwiGLU에서는 입력을 **두 갈래로 변환한 뒤 원소별로 곱하는 gate 구조**를 사용.

- `\sigma`: 활성화 함수.
- `\odot`: 원소별 곱.
- 한 갈래의 활성화 결과가 다른 갈래의 정보를 조절.
- 마지막 선형 변환으로 다시 D차원 출력.

파라미터 수를 맞추려면

- 기존: **D** → **4D** → **D**, 가중치 행렬 2개.
- SwiGLU: **D** → **H** 변환 2개와 **H** → **D** 변환 1개.
- H = 8D/3 ⇒ 기존 MLP와 가중치 파라미터 수가 같아짐.

11-4. Mixture of Experts — MoE

블록 안에 서로 다른 MLP를 E개 준비. 각 MLP가 하나의 expert.

- 각 expert는 별도의 가중치를 가짐.
- 토큰마다 router가 **E개 중 A개 expert를 선택**.
- 선택된 expert가 해당 토큰을 처리.
- **A < E**이므로, 모든 expert를 매번 실행하지 않음.

입력 토큰 → Router → 일부 Expert MLP → 출력

기존 MLP 하나를 기준으로:

⇒ **전체 파라미터 수를 크게 늘리면서, 토큰당 사용하는 계산량의 증가는 제한**.

항목	Expert 확장에 따른 변화
MLP 파라미터 수	약 **E배**
토큰당 MLP 계산량	약 **A배**

개선 방향 정리

변경	바꾸는 부분
Pre-Norm	정규화 위치
RMSNorm	정규화 계산 방식
SwiGLU	MLP의 내부 구조
MoE	MLP를 여러 expert로 확장하고 선택적으로 사용

강조점

1. **Attention의 기본 흐름:** 관련성 점수 → softmax → Value 가중합.
2. **Query·Key·Value의 역할:** 찾을 정보의 기준 / 관련성 비교 대상 / 실제로 가져올 정보.
3. **Self-Attention:** 같은 입력에서 Q·K·V를 만들고, 입력 사이의 정보를 교환.
4. **위치 정보와 마스킹은 역할이 다름:** Positional Encoding은 위치 제공, masking은 참조 범위 제한.
5. **Multihead:** 서로 다른 가중치의 Attention을 병렬 실행하고, 출력을 결합.
6. **긴 입력의 비용:** 기본 Attention 행렬은 제곱 크기. FlashAttention은 전체 행렬 저장을 피해 메모리 부담을 줄임.
7. **Transformer Block의 역할 분담:** Attention은 벡터 사이, MLP·LayerNorm은 각 벡터 내부를 처리.
8. **언어와 이미지의 적용:** 언어는 토큰과 인과적 마스킹, ViT는 이미지 패치와 전체 패치 참조.
9. **구조 개선:** 정규화 위치·방식, MLP 구성, expert 선택을 바꾸며 기본 블록을 확장.